

Lab 9

Lab 8 introduced `SpriteSheet` as a flexible animation container, but creating one still meant writing C# code — listing every animation, frame range, and duration in the constructor of whoever spawned the object. Lab 9 finishes the abstraction by moving sprite sheet definitions into JSON files, then puts the new system to immediate use: the player gains directional walking animations that switch automatically based on movement input.

JSON sprite sheet definitions

Two new files appear under `Assets/`. The simpler one, `BombExploding.json`, just captures what was previously hard-coded inside `Engine.AddBomb`:

```
{
  "rowCount": 1,
  "columnCount": 13,
  "frameWidth": 32,
  "frameHeight": 64,
  "frameCenter": {
    "offsetX": 16,
    "offsetY": 48
  },
  "fileName": "BombExploding.png",
  "animations": {
    "Explode": {
      "startFrame": { "row": 0, "col": 0 },
      "endFrame": { "row": 0, "col": 12 },
      "durationMs": 2000,
      "loop": false
    }
  }
}
```

Every parameter that used to be a constructor argument is now a JSON field. The image filename is part of the document, so a single `Load(...)` call resolves both the texture and the metadata.

`Player.json` shows where the new format pays off — five animations defined declaratively, no code changes required to add or tweak them:

```
{
  "rowCount": 10,
  "columnCount": 6,
  "frameWidth": 48,
  "frameHeight": 48,
  "frameCenter": { "offsetX": 24, "offsetY": 42 },
  "fileName": "player.png",
  "animations": {
    "IdleDown": { "startFrame": { "row": 0, "col": 0 }, "endFrame": { "row": 0,
```

```

"col": 5}, "durationMs": 1000, "loop": true },
  "MoveLeft": { "startFrame": {"row": 4, "col": 0}, "endFrame": {"row": 4,
"col": 5}, "durationMs": 1000, "loop": true, "flip": 1 },
  "MoveRight": { "startFrame": {"row": 4, "col": 0}, "endFrame": {"row": 4,
"col": 5}, "durationMs": 1000, "loop": true, "flip": 0 },
  "MoveDown": { "startFrame": {"row": 3, "col": 0}, "endFrame": {"row": 3,
"col": 5}, "durationMs": 1000, "loop": true, "flip": 0 },
  "MoveUp": { "startFrame": {"row": 5, "col": 0}, "endFrame": {"row": 5,
"col": 5}, "durationMs": 1000, "loop": true, "flip": 0 }
}
}

```

A few things to notice. `MoveLeft` and `MoveRight` reuse the same row from the sprite sheet — the only difference is `flip`, where `1` means horizontal flip (`RendererFlip.Horizontal`) and `0` means none. The artist drew the walking animation once, facing right, and the engine produces the leftward variant for free. The `IdleDown` animation loops over six frames at one frame per ~167ms, giving the character a subtle breathing motion when standing still.

SpriteSheet — JSON-friendly types and a Load method

`SpriteSheet` is restructured to be deserializable by `System.Text.Json`. The two main changes are nested types and property accessibility.

The tuple-based fields from Lab 8 are replaced with named structs, because `System.Text.Json` cannot deserialize tuples directly:

```

public struct Position
{
    public int Row { get; set; }
    public int Col { get; set; }
}

public struct Offset
{
    public int OffsetX { get; set; }
    public int OffsetY { get; set; }
}

public class Animation
{
    public Position StartFrame { get; set; }
    public Position EndFrame { get; set; }
    public RendererFlip Flip { get; set; } = RendererFlip.None;
    public int DurationMs { get; set; }
    public bool Loop { get; set; }
}

```

All `init` properties throughout the class are also relaxed to `set`. JSON deserialization assigns properties one at a time after construction, which is incompatible with `init`-only setters that lock after the

constructor runs.

The constructor that took explicit parameters is gone, replaced by a static `Load` method that reads the JSON, validates the result, and loads the referenced texture:

```
public static SpriteSheet Load(GameRenderer renderer, string fileName, string
directory)
{
    var json = File.ReadAllText(Path.Combine(directory, fileName));
    var spriteSheet = JsonSerializer.Deserialize<SpriteSheet>(json, new
JsonSerializerOptions
    {
        PropertyNameCaseInsensitive = true,
    });

    if (spriteSheet == null)
    {
        throw new Exception($"Failed to load sprite sheet: {fileName}");
    }

    if (spriteSheet.FileName == null)
    {
        throw new Exception($"Sprite sheet {fileName} does not have a file
name.");
    }

    if (spriteSheet.FrameWidth <= 0 || spriteSheet.FrameHeight <= 0)
    {
        throw new Exception($"Sprite sheet {fileName} has invalid frame
dimensions.");
    }

    if (spriteSheet.RowCount <= 0 || spriteSheet.ColumnCount <= 0)
    {
        throw new Exception($"Sprite sheet {fileName} has invalid row/column
count.");
    }

    spriteSheet._textureId = renderer.LoadTexture(Path.Combine(directory,
spriteSheet.FileName), out _);
    if (spriteSheet._textureId == -1)
    {
        throw new Exception($"Failed to load texture for sprite sheet:
{spriteSheet.FileName}");
    }

    return spriteSheet;
}
```

`PropertyNameCaseInsensitive = true` lets the JSON use camelCase (`frameWidth`) while the C# properties are PascalCase (`FrameWidth`). Each validation check guards against malformed or missing data

with a descriptive error.

The `Render` method itself is unchanged from Lab 8 — the rendering logic was already separated cleanly from the construction logic, so only the entry point needed updating.

Engine — using the loader

Every site that previously built a sprite sheet by hand now goes through `SpriteSheet.Load`. The change in `AddBomb` is the most dramatic, since Lab 8's version had a long block of inline animation setup:

```
private void AddBomb(int screenX, int screenY)
{
    var worldCoords = _renderer.ToWorldCoordinates(screenX, screenY);

    SpriteSheet spriteSheet = SpriteSheet.Load(_renderer, "BombExploding.json",
"Assets");
    spriteSheet.ActivateAnimation("Explode");

    TemporaryGameObject bomb = new(spriteSheet, 2.1, (worldCoords.X,
worldCoords.Y));
    _gameObjects.Add(bomb.Id, bomb);
}
```

The body shrinks to four real lines. All of the bomb's parameters — frame dimensions, animation timing, frame center, image path — now live in the JSON file. Tuning the explosion no longer requires recompiling the project.

The player is created the same way, with starting coordinates supplied directly:

```
_player = new(SpriteSheet.Load(_renderer, "Player.json", "Assets"), 100, 100);
```

The renderer's camera follow code is updated to read from the player's inherited `Position` tuple instead of the old `X / Y` properties:

```
var playerPosition = _player!.Position;
_renderer.CameraLookAt(playerPosition.X, playerPosition.Y);
```

This is a consequence of `PlayerObject` becoming a proper `RenderableGameObject` — see below.

The `UpdatePosition` call also gains two extra arguments:

```
_player?.UpdatePosition(up, down, left, right, 48, 48, msSinceLastFrame);
```

The `48, 48` represent the player sprite's frame width and height. (They are passed in for future use —

clamping the player to the world bounds, for example — but the current implementation does not yet read them.)

PlayerObject — animated and integrated

`PlayerObject` undergoes the biggest behavioural change in this lab. It now extends `RenderableGameObject` instead of `GameObject`, which means it inherits `Position`, `SpriteSheet`, `Render`, and ID generation — and the custom code that did all of those things by hand is deleted.

```
public class PlayerObject : RenderableGameObject
{
    private const int _speed = 128; // pixels per second
    private string _currentAnimation = "IdleDown";

    public PlayerObject(SpriteSheet spriteSheet, int x, int y) :
base(spriteSheet, (x, y))
    {
        SpriteSheet.ActivateAnimation(_currentAnimation);
    }
}
```

The constructor takes a sprite sheet (loaded externally by the caller, from `Player.json`) and a starting position, and immediately activates the idle animation. There is no texture loading, no source/destination rectangles, no `Render` override — all of that comes from the base class.

The `UpdatePosition` method gains animation switching alongside its movement logic:

```
public void UpdatePosition(double up, double down, double left, double right,
int width, int height, double time)
{
    if (up + down + left + right == 0)
    {
        return;
    }

    var pixelsToMove = _speed * (time / 1000.0);

    var x = Position.X + (int)(right * pixelsToMove);
    x -= (int)(left * pixelsToMove);

    var y = Position.Y + (int)(down * pixelsToMove);
    y -= (int)(up * pixelsToMove);
}
```

The early return when no direction is pressed is a small but meaningful change — if the player is standing still, nothing needs to be recalculated, and no spurious animation switching happens.

After computing the new coordinates, the method picks an animation based on which direction the player is moving:

```

var newAnimation = _currentAnimation;

if (y < Position.Y && _currentAnimation != "MoveUp")
{
    newAnimation = "MoveUp";
}

if (y > Position.Y && newAnimation != "MoveDown")
{
    newAnimation = "MoveDown";
}

if (x < Position.X && newAnimation != "MoveLeft")
{
    newAnimation = "MoveLeft";
}

if (x > Position.X && newAnimation != "MoveRight")
{
    newAnimation = "MoveRight";
}

if (x == Position.X && y == Position.Y && newAnimation != "IdleDown")
{
    newAnimation = "IdleDown";
}

if (newAnimation != _currentAnimation)
{
    _currentAnimation = newAnimation;
    SpriteSheet.ActivateAnimation(_currentAnimation);
}

Position = (x, y);
}

```

The cascade of `if` statements walks through the four cardinal directions and falls back to `IdleDown`. Because each check is an independent `if` (not `else if`), later checks can overwrite earlier ones — which means horizontal movement takes priority over vertical (left/right is checked after up/down, so a diagonal walk plays the horizontal animation). This is a typical 2D RPG convention.

The animation is only re-activated when it actually changes — guarding with `newAnimation != _currentAnimation` prevents the sprite sheet from constantly resetting its animation timer, which would freeze the character on the first frame as long as the player held a direction key. This is exactly the kind of subtle bug that the lab's design avoids by tracking the current animation name as state.

The final `Position = (x, y)` writes the new position back to the inherited tuple. The base class's `Render` method handles drawing automatically on the next frame.

Summary

By the end of Lab 9, sprite sheets are fully data-driven — both the bomb explosion and the player character are described in JSON files that can be edited without touching any C# code. The player itself is no longer a special case in the type hierarchy; it inherits everything from `RenderableGameObject` and adds only the small bit of logic that distinguishes it from any other animated thing in the world. Walking around now produces directional animations that switch smoothly with movement and return to an idle loop when standing still, while the underlying engine code becomes shorter and cleaner at every call site that touches sprites.